Timeout

An NBA play recommender

Pause a basketball game at any moment. Timeout looks at where all ten players and the ball are, and draws on the frame the best thing the offense could do next — an arrow, a highlighted player, and a number telling you how many points that choice is worth. Then it explains why, in a sentence.

one State schema — identical from broadcast CV or SportVU tracking

A complete guide — read this and you understand the whole project.

1. What Timeout is

Timeout is a tool that watches basketball footage and acts like an assistant coach who never blinks. Whenever you pause, it answers one question: **of everything the player with the ball could do right now — shoot, drive, pass to a teammate, set a screen — which is worth the most?**

The important idea is what it is *not*. It is not trying to predict what *will* happen. It is grading the *options*. Think of it as a second opinion with a number attached: not “he is going to pass left,” but “a pass to the corner is worth about 1.1 points; a contested shot here is worth about 0.8, so the pass is the better play.”

Everything it shows is measured, not guessed. Player positions come from the video. The scores come from models trained on thousands of real NBA possessions. The written explanation is only allowed to repeat numbers the model already produced — it can’t make things up. And when the picture is too unclear to be sure (a replay, a close-up, players hidden behind each other), it says so and shows nothing rather than guess.

The one formula, in plain words

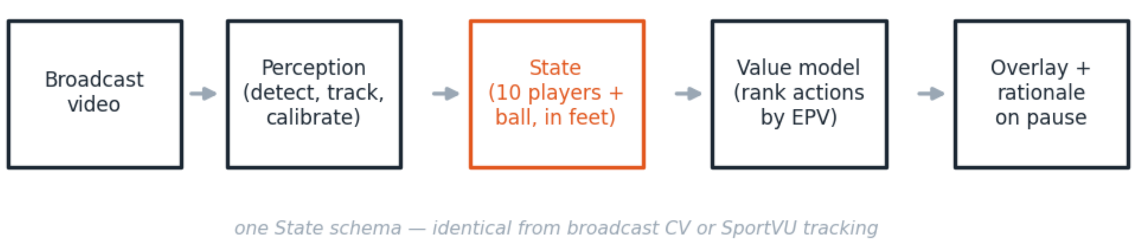
Every option is scored the same way. The value of an action is: how likely it is to succeed, times how good the resulting situation is — plus, for the times it fails, how bad the turnover is. Written out:

```
value(action) = P(success) × value(good outcome) + (1 − P(success)) ×
value(turnover)
```

“Value” here is measured in **expected points** — the average number of points that situation is worth. This single number lets you compare a shot, a pass, and a drive on the same scale, which is the whole point.

2. How it works, end to end

The system is a pipeline: raw video goes in one end, a drawn-on recommendation comes out the other. There are four stages.



The pipeline. The middle box — the State — is the heart of the design.

Stage 1 — Perception. Look at the video and figure out where everyone is. Detect the players and the ball, follow them frame to frame, and convert their positions from “pixels on screen” into “feet on the court.”

Stage 2 — The State. Package all of that into one tidy object: ten players and the ball, in court coordinates, with their speeds, who is guarding whom, how much space there is, and a confidence score. This object is the contract the rest of the system depends on.

Stage 3 — The value model. Given the State, list every sensible action the ball-handler could take and score each one in expected points.

Stage 4 — The app. Draw the best action on the paused frame, show the ranked list of alternatives, and — on request — explain the choice in plain English or answer questions in a chat box.

Why the State matters so much

The clever part of the design is that the State looks *exactly the same* whether it came from broadcast video or from the NBA’s official player-tracking data (called SportVU). Because both sources produce an identical State, the scoring models — which were trained on the clean official data — work without any changes on messy video. The hard, unreliable part (reading video) is walled off from the smart part (scoring), and they meet at one clean hand-off.

3. Seeing the game (perception)

Turning a video into court positions is the messiest job in the project. Here is each piece, in order.

Calibration — teaching it where the court is

A camera can be anywhere, at any angle. To convert screen pixels into real court feet, the system needs to know how the court is positioned in the shot. You do this once per camera angle: a small court diagram highlights a landmark (say, the corner of the paint), and you click the matching spot in the video. After 5–8 clicks it works out the full mapping (a *homography*) and draws the court lines back onto the frame so you can check the fit. As the camera pans, the mapping is carried along automatically by tracking the movement between frames.

Calibrated shot	Landmarks clicked	Accuracy (reprojection error)
early_a.json	8	0.842 ft
early_b.json	5	0.144 ft
calib.json	7	1.325 ft

Reprojection error is how far off the mapping is when checked against the clicked points — under about 1.5 ft is good; these are all well within that.

Detection — finding players and the ball

A detector scans each sampled frame and boxes the players and the ball. The big lesson learned here: a generic “find people” detector is the wrong tool, because it also boxes the crowd, the bench, coaches, and referees — sometimes 30–40 “people” when only 10 are playing. The project now uses a purpose-built basketball detector (a hosted Roboflow workflow) with a dedicated *player* class that ignores the crowd, a separate *referee* class so officials can be dropped, and reliable *ball* detection. On the test clip this took the count from a noisy 30+ down to a clean 10 on-court players.

Following, teams, and names

- **Tracking:** each player keeps the same identity from frame to frame, so paths are smooth and the ball-handler doesn’t flicker between people.
- **Teams:** jersey colours are clustered into two groups, so the system knows which five are on offense and which five are defending.
- **Jersey numbers:** an optical-character-reader reads the numbers it can see and votes across the whole shot, so a player becomes “#7” and the explanations can name him. Players facing away stay unnamed rather than being guessed.
- **Ball tracking over time:** the ball is small and blurs, so it is missed on some frames. Instead of only trusting a single frame, the system models the ball’s motion and coasts its position through the gaps, so every pause has a ball — and therefore a known handler.

The confidence gate — knowing when to stay quiet

Every State carries a confidence score built from how many players were tracked, how good the court mapping is, and whether the ball was seen. Below a threshold the app withholds the recommendation and tells you why (replay, close-up, occlusion, or a stretch of video that wasn’t calibrated). Showing nothing is treated as better than showing a wrong arrow.

4. Scoring the options (the value model)

Once there is a clean State, the system lists the legal actions for the ball-handler (up to about 13 of them: shoot, drive left/right, pass to each teammate, set or use a screen, reset) and scores each with the formula from section 1. Three ingredients feed that formula.

The three sub-models

- **Shot model** — how likely a shot is to go in, given who is shooting, from where, and how closely guarded. It blends a per-player shooting history with the situation.
- **Pass model** — how likely a pass is to reach its target without being stolen.
- **Drive model** — how likely a drive to the basket gets there successfully.

These three are gradient-boosted models (LightGBM) — fast, reliable classifiers trained on real NBA outcomes.

The situation value, V(s)

The formula also needs the value of the *resulting* situation — e.g. how good it is to have the ball in the corner with a step of space. That is a neural network called V(s). It is built to be *order-independent*: it treats the players as a set, so shuffling their order can't change the answer. It runs on a GPU when one is available.

Two honesty rules

- **Actions are objects, never coordinates.** The model outputs "pass to the player in the left corner," and only the renderer turns that into an arrow — and the arrow's endpoints are always real tracked positions, never invented ones.
- **Never invent a missing defender.** If a defender can't be seen, the system lowers its confidence instead of imagining one, and may withhold the recommendation.

5. The web app

The product is a local web page with two parts.

The studio (setup wizard)

You upload a clip from your computer or paste a YouTube link. It grabs the footage, opens a calibration step where you click the court landmarks (and can click where the ball is in the first frame to seed tracking), and then builds the recommendations. You can calibrate several camera angles to cover more of the game.

The player

- Play the footage and **pause anywhere**; the best action is drawn on the frame with an arrow, a highlighted player, and its expected-points number.
- A ranked list of **alternative actions** sits beside the video — click any to draw it instead.
- Ask **why** and read a one-paragraph rationale for the chosen action.
- A **chat assistant** answers free-form questions ("why not shoot?", "show the drive", "pass to #7") and can change the drawn play. It runs through a language model (Groq or Claude) when a key is set, and falls back to built-in rules otherwise.

Crucially, the app does **no live inference**. All the heavy work is done once, up front, and saved to a file the page reads. So pausing is instant.

Because each drawn overlay is computed for its own frame, the app refuses to reuse one recommendation on a far-away moment — that is what previously made an arrow appear frozen and point into the stands during an uncalibrated opening stretch. It now shows an honest "outside the calibrated segment" note there instead.

6. How the models were trained

The scoring models learn from two kinds of data.

- **Synthetic data** — a built-in generator makes fake-but-realistic possessions with a known "right answer," so the whole pipeline can be tested and measured with no downloads and no GPU.
- **Real data** — NBA 2015-16 SportVU player-tracking joined to real shot and play-by-play outcomes. The models here were trained on roughly **240 games** (about 25,000 shots, 45,000 passes, 49,000 drives, and 600,000+ situation snapshots), split so that test games are never seen during training.

The possession parser recovers realistic NBA rates on its own — about 100 possessions per team per game at roughly one point per possession — which is a good sign the inputs are sane before any model is trained.

7. Benchmark results

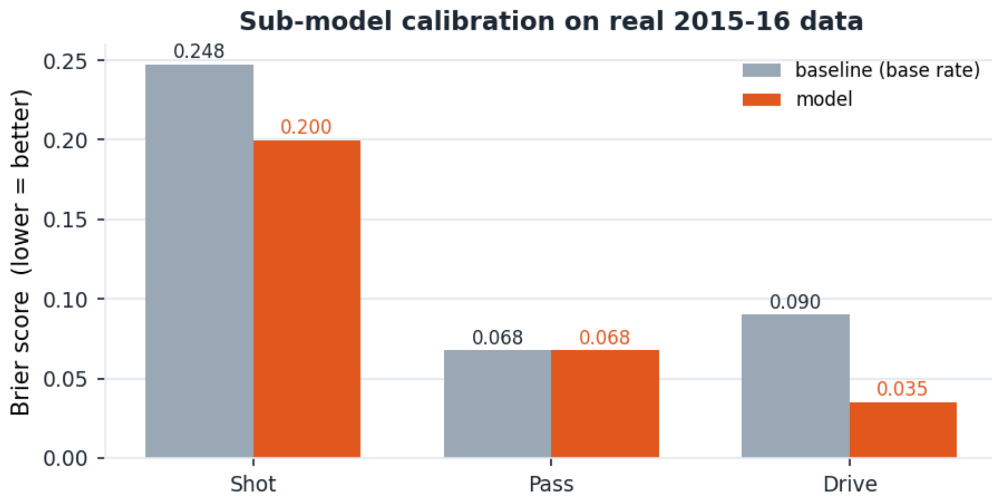
Every number below was produced by the project's own scripts (`scripts/benchmark.py` and the training runs).

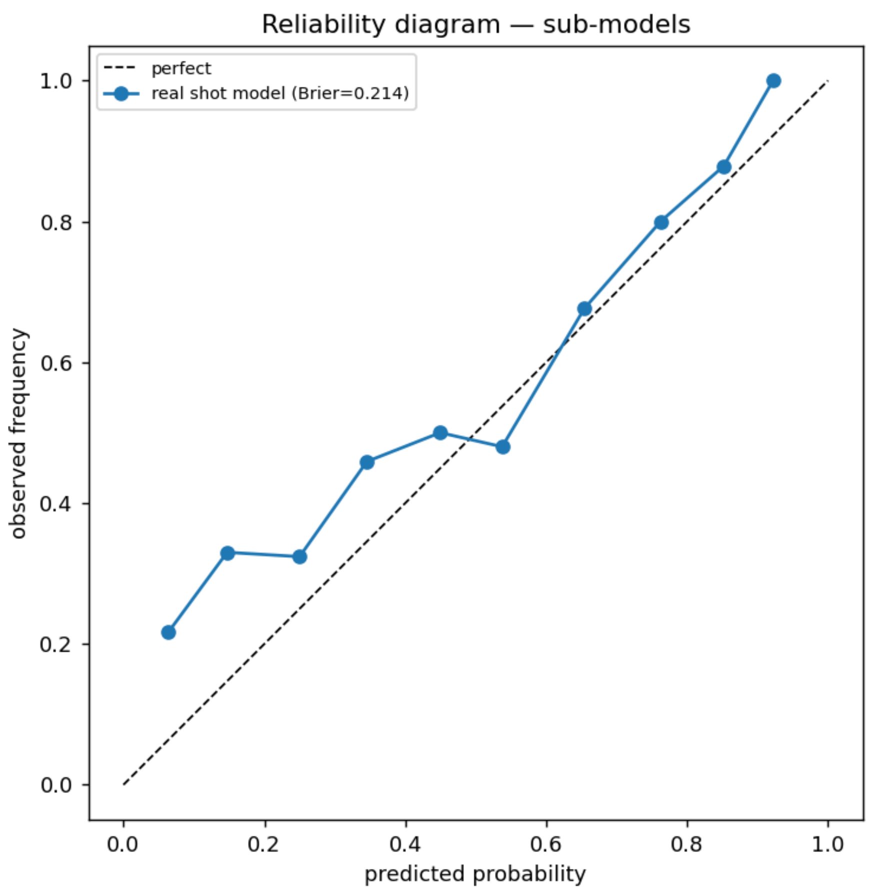
Does the scoring actually work? (real 2015-16 data)

Sub-model	Score (Brier)	Baseline	Verdict
Shot	0.200	0.248	beats baseline
Pass	0.068	0.068	ties baseline
Drive	0.035	0.090	beats baseline

Brier score measures how well-calibrated a probability is — lower is better, and the baseline is "just guess the average rate every time." The shot and drive models clearly beat that; pass completion ties it, because passes succeed ~90%

of the time and the flat guess is already hard to beat.





Shot-model reliability: predicted make-probability vs. what actually happened. Closer to the diagonal is better.

Does it pick good actions? (simulated, vs. naive strategies)

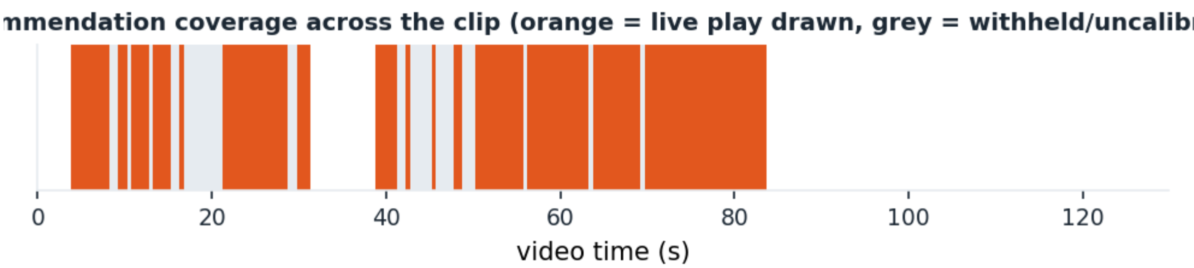
Strategy	Right or defensible	Clearly wrong	Avg. points lost vs. best

Timeout	86%	4%	0.058
Always pass to most open	40%	48%	0.274
Always shoot	30%	55%	0.337
Random	20%	55%	0.352

Against a known ground-truth generator, Timeout's top pick is right or defensible about 86% of the time and clearly wrong only 4%, losing far fewer expected points than any simple rule.

Does it work on real footage? (the test clip)

What was measured	Result
Clip	[1280, 720] at 30.0 fps, 129.8 s
Pause points with a recommendation	111 of 146 (76.0%)
On-court players detected per pause	median 10 (range 7–13)
Pauses with a live ball detection	90.1%
Pauses with a resolved ball-handler	100.0%
Pauses with the rim located	100.0%



A median of exactly 10 players per pause is the headline: it means the detector is seeing the five-on-five action and not the crowd.

Is it fast enough?

Scoring one paused moment — listing all 13 candidate actions and running every model — takes about **37.83 ms** (median; 41.51 ms at the 90th percentile). The design budget was 2 seconds, so there is enormous headroom, and because results are pre-computed the app itself pauses instantly.

And the whole system is covered by an automated test suite: **83 tests pass** end to end.

8. Honest limitations

A description isn't complete without the rough edges. These are known and documented, not hidden.

- **The situation model V(s) is weak on real data.** The shot/pass/drive models are solid, but predicting the exact value of a single state from single-sample outcomes is hard; V(s) barely beats a flat average. It needs smoothed multi-step returns and more data to improve. The action *ranking* still works because it leans mostly on the three strong sub-models.
- **Calibration is manual.** You click court landmarks once per camera angle. An automatic court-line detector was tried and set aside — broadcast courts mix light and dark lines with logo and jersey interference, so it wasn't reliable. A trained keypoint model is the eventual fix.
- **Coverage has gaps.** Recommendations exist only inside calibrated camera shots; between them (and after a cut) the app honestly shows nothing rather than a stale arrow. More calibrations = more coverage.
- **The action mix can skew.** On the test clip the top pick is "set a screen" most of the time; whether that is genuinely optimal or a model bias is the next thing to investigate — a scoring question, not a detection one.

9. Running it yourself

Install and run the test suite (no data or GPU needed):

```
pip install -e .
python -m pytest -q
```

Build and watch the web app on a clip:

```
# calibrate one frame per camera angle (a court diagram guides your clicks)
python scripts/calibrate.py --video clip.mp4 --time 39 --out calib.json
# pre-compute recommendations, then serve the player
python scripts/build_webapp.py --video clip.mp4 --shots calib.json
python scripts/serve_webapp.py # open the printed URL
```

For the best detection, set a Roboflow key (read from the environment, never stored) and add `--detector roboflow`. For the chat assistant, set a Groq or Anthropic key. Re-run the whole benchmark with `python scripts/benchmark.py`.

10. Glossary

Term	Plain meaning
Expected points (EPV)	The average points a situation or action is worth — the common yardstick that lets a shot, pass, and d
State	The tidy object describing all ten players and the ball in court feet, plus speeds, matchups, spacing, ar
Homography	The mathematical mapping that converts a point on the screen into a point on the real court, worked c
Sub-model	One of the three trained classifiers — shot, pass, or drive — that estimates the chance an action succ
V(s)	The neural network that estimates how valuable a resulting situation is.
Brier score	A measure of how well-calibrated a predicted probability is; lower is better.

Confidence gate	The rule that withholds a recommendation when the picture is too unclear to trust.
SportVU	The NBA’s official optical player-tracking data, used to train the models.

Timeout — half-court offense, ball-handler decisions, one honest recommendation per pause.